Vietnam National University - Ho Chi Minh City
University of Science
Faculty of Information Technology

SEMINAR DOCUMENT

PROGRESS REPORT

Team Members

Name	ID	Email
Cao Uyển Nhi	22127310	cunhi22@clc.fitus.edu.vn
Lưu Thanh Thuý	22127410	ltthuy22@clc.fitus.edu.vn
Nguyễn Phước Minh Trí	22127424	npmtri22@clc.fitus.edu.vn
Võ Lê Việt Tú	22127435	vlvtu22@clc.fitus.edu.vn
Trần Thị Cát Tường	22127444	ttctuong22@clc.fitus.edu.vn

Supervisors

Name
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

June 16, 2025

Contents

Revision History

Version	Date	Author	Description
1.0	2025-06-10	Cao Uyển Nhi	Initial template creation and project structure setup
1.1	2025-06-12	Cao Uyển Nhi	Added content structure, sections, and project planning
1.2	2025-06-14	Trần Thị Cát Tường	Added JMeter architecture and components research
1.3	2025-06-14	Lưu Thanh Thuý	Added basic performance testing concepts and methodology
1.4	2025-06-16	Nguyễn Phước Minh Trí	Completed performance testing tools comparison research
1.5	2025-06-17	Võ Lê Việt Tú	Added JMeter installation guide and environment setup documentation
1.6	2025-06-19	Cao Uyển Nhi	Added AI applications in performance testing research
1.7	2025-06-19	Lưu Thanh Thuý	Completed business flow analysis for JPetStore system
1.8	2025-06-20	Trần Thị Cát Tường	Designed test scenarios and test plan for JMeter implementation
1.9	2025-06-21	Cao Uyển Nhi	Consolidated research findings and prepared final report structure

Chapter 1

Introduction

1.1 Project Overview

The Performance Testing Tools seminar project is a comprehensive study of performance testing tools, focusing on Apache JMeter and AI applications in performance testing. The project aims to explore the basic concepts of Performance Testing (Load Testing, Stress Testing, Spike Testing, Endurance Testing), compare popular tools in the market, and conduct a practical demo with JMeter on the JPetStore system.

The importance of this project is shown by how Performance Testing plays a key role in ensuring software quality, especially in the digital age when web applications must serve millions of users at the same time. Research and application of AI in Performance Testing is also a new trend, helping to automate result analysis and detect system issues.

1.2 Project Objectives

This project aims to achieve the following key objectives:

- **Understand Core Concepts**: To research and clearly define the fundamental principles of Performance Testing, including its primary types: Load, Stress, Spike, and Endurance Testing.
- **Compare Testing Tools**: To conduct a comparative analysis of popular performance testing tools, evaluating their features, strengths, and weaknesses to identify suitable use cases.
- **Deep Dive into JMeter**: To gain in-depth knowledge of the Apache JMeter tool, covering its architecture, core components, and practical usage.
- **Investigate AI Applications**: To explore how Artificial Intelligence (AI) can be applied to enhance performance testing, particularly in areas like automated result analysis and anomaly detection.
- **Conduct a Practical Demo**: To apply theoretical knowledge by designing and executing performance test scenarios on the JPetStore demo application using JMeter.
- **Document and Report Findings**: To produce a comprehensive report detailing the research process, test results, and conclusions drawn from the project.
- **Share Knowledge**: To present the project's findings and practical demonstrations to the instructor and fellow students, contributing to the collective understanding of the topic.

1.3 Scope of the Project

1.3.1 Within project scope

- Research theory about Performance Testing (Load, Stress, Spike, Endurance Testing)
- Compare Performance Testing tools: JMeter, NeoLoad, WebLOAD, LoadUI, LoadRunner
- Deep research on Apache JMeter (architecture, components, installation)
- AI applications in Performance Testing (anomaly detection, automated analysis)
- Conduct demo on JPetStore system
- Design and execute test scenarios for business flow: Register → Login → Purchase → Payment

1.3.2 Outside project scope

- No deep research on Security Testing or other testing types
- No development of new automated tools
- No testing on real production systems
- No coverage of performance testing for mobile applications

1.4 Team and Stakeholders

Role	Member	Responsibility
Team Leader	Cao Nhi	Planning, AI research in Performance Testing, analysis evaluation, presentation MC
Member	Thanh Thuý	Basic research on Performance Testing and business flow analysis
Member	Minh Trí	Performance Testing tool comparison and JMeter test script writing
Member	Cát Tường	Deep JMeter research (theory) and test plan design
Member	Việt Tú	JMeter installation setup, test execution, and demo video recording

Stakeholder Type	Stakeholder	Interest/Expectation
Instructor	Course instructor	Evaluate research quality, practical application, presentation skills
Audience	Other student groups in class	Learn new knowledge about Performance Testing, reference research methods
Research Community	Software Testing research community	Scientific value of research, knowledge contribution about AI in Performance Testing

1.5 Timeline Summary

The project is divided into 3 main phases over 9 weeks:

Phase	Timeline	Activities	Status
Phase 1 - Foundation & Research	Week 1-3	Complete theoretical research, tool comparison, JMeter environment setup	`Done`

Phase	Timeline	Activities	Status
Phase 2 - Scenario Design & Testing	Week 4-7	Select test system (JPetStore), design test scenarios, write JMeter scripts, execute testing and apply AI	`About to start`
Phase 3 - Reporting & Presentation	Week 8-9	Prepare slides, write detailed report, record demo video and conduct presentation	`Not started yet`

Currently the team is at the end of Phase 1, preparing to move to Phase 2 with test system selection and business flow analysis.

Chapter 2

Phase 1: Foundation & Research

2.1 Planning

2.1.1 Phase 1: Foundation & Research (Week 1-3)

Task	Content	Output	Week	Assignee
Plan the Seminar	Set goals, topic, and team roles	Plan file	Week 1	Cao Nhi
Learn Performance Testing basics	Load, Stress, Spike, Endurance Testing	Summary notes	Week 2	Thanh Thuý
Research AI in Performance Testing	e.g., AI for finding issues, adjusting load	AI use case description	Week 2	Cao Nhi
Compare Performance Testing tools	JMeter, NeoLoad, WebLOAD, LoadUI, LoadRunner	Comparison table	Week 2	Minh Trí
Study JMeter theory	Architecture, parts: Thread Group, Sampler, Listener…	Internal guide document	Week 3	Cát Tường
Install & set up JMeter	Set up JMeter, test a sample plan	Setup video + test script	Week 3	Việt Tú

2.1.2 Phase 2: Scenario Design & Testing (Week 4-7)

Task	Content	Output	Week	Assignee
Choose a system to test	JPetStore	System description	Week 4	Cao Nhi
Analyze the test flow	Sign up → Log in → Buy → Checkout	Flowchart (draw.io/pdf)	Week 4	Thanh Thuý
Design test plan and test cases	Write test cases; set test goals (users, time)	Test Plan file	Week 5	Cát Tường
Write JMeter test scripts	Create scripts with HTTP Request, Assertion, Listener, Timer…	.jmx file	Week 5	Minh Trí
Run tests and get results	Record metrics: response time, throughput, error rate…	Results table (.csv), charts	Week 6	Việt Tú

Task	Content	Output	Week	Assignee
Advanced: Use AI	Auto-analyze logs, alert issues	Code/script, prompt	Week 7	Cao Nhi

2.1.3 Phase 3: Reporting & Presentation (Week 8-9)

Task	Content	Output	Week	Assignee
Prepare presentation slides	Content: theory → tool → demo → conclusion	.pptx file	Week 8	All
Report on theory	Explain concepts, tools, and why we chose them	Section I in the Report	Week 8	Thanh Thuý
Report on practice (Scenario)	Describe the test scenario	Section II, part 1 in the Report	Week 8	Cát Tường
Report on practice (Testing)	Describe the test process and results	Section II, part 2 in the Report	Week 8	Minh Trí
Analysis & Review	Analyze results, limits, and suggestions	Section III in the Report	Week 8	Cao Nhi
Record a demo video	Run test + show results report	Video .mp4 (3–5 mins)	Week 8	Việt Tú
Presentation	Theory, Scenario, Process, Video demo		Week 9	MC (Cao Nhi) + All

2.2 Performance Testing Overview

2.2.1 Non-functional Testing

Non-Functional Testing is the process of evaluating aspects of the software that are not related to its functionality. Instead of focusing on testing specific features, it concentrates on factors such as performance, security, reliability, user interaction, and compatibility. The goal of non-functional testing is to ensure that the software performs well not only from a functional perspective but also from other aspects.

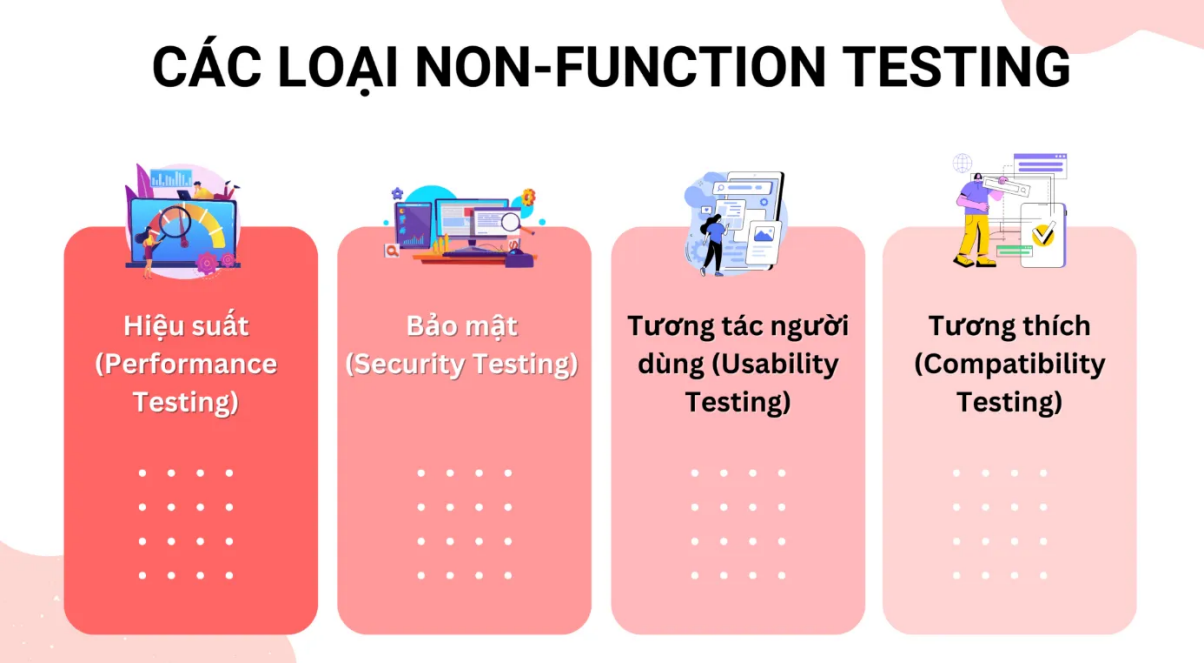

Figure 2.1. Types of non-functional testing.

2.2.2 What is Performance Testing?

Performance Testing is the process of checking whether a software program runs fast, responds quickly, handles load well, how it uses resources (CPU, RAM, bandwidth, etc.), and whether it can scale to accommodate many concurrent users.

Simply put, the goal of performance testing is to detect and eliminate points that cause the software to run slow, become congested, or fail to meet desired performance levels. Performance testing focuses on the **speed and stability** of the software in real-world use, rather than testing the correctness of its functionality.

The focus of **Performance Testing** is to check for issues in the software program such as:

- **Speed** - Determines if the application responds quickly.
- **Scalability** - Determines the maximum user load the software application can handle.
- **Stability** - Determines if the application is stable under varying loads.

Performance Testing is often used to help identify bottlenecks in a system, establish a baseline for future testing, support effective performance tuning, determine compliance with performance goals and requirements, and gather other relevant operational data to help stakeholders make decisions related to the overall quality of the applications being tested. Additionally, the results from performance testing and analysis can help you estimate the hardware configuration needed to support the applications when you release the product for widespread use.

2.2.3 Types of Performance Testing

Load testing

Load testing is performed to evaluate the system's behavior under increasing load (number of concurrent users/CCU, number of transactions/requests, etc.), to **determine the maximum load (threshold)** that the system can handle.

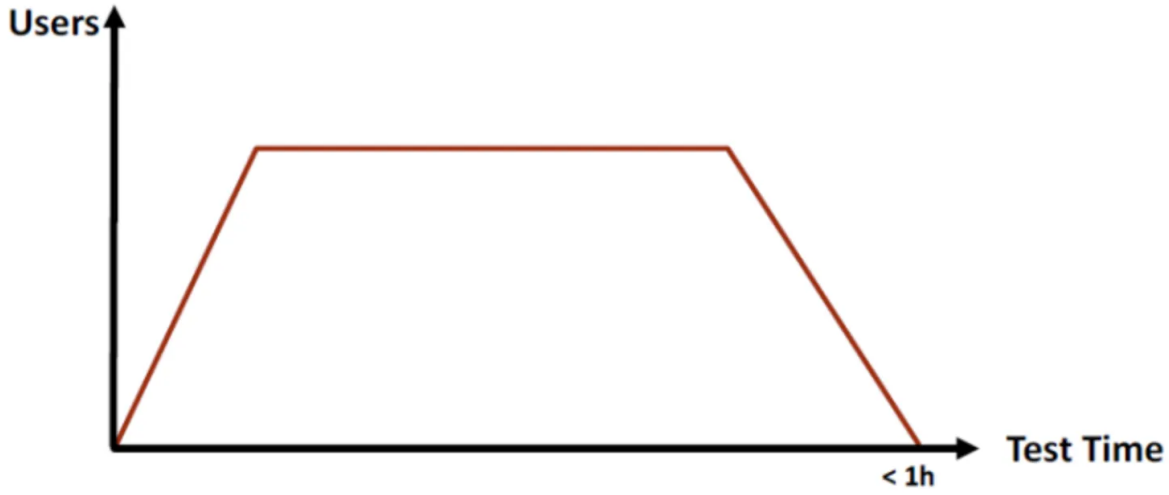

Figure 2.2. Load Testing

A tip to find the threshold quickly is to double the number of concurrent requests sent to the system with each test. For example:

- Test 1: Send 3000 concurrent requests
- Test 2: Send 6000 concurrent requests
- Test 3: Send 12000 concurrent requests

If the system becomes overloaded in Test 3, then in Test 4, reduce the number of requests added in Test 3 by half, meaning:

- Test 4: Send 9000 concurrent requests

Stress testing

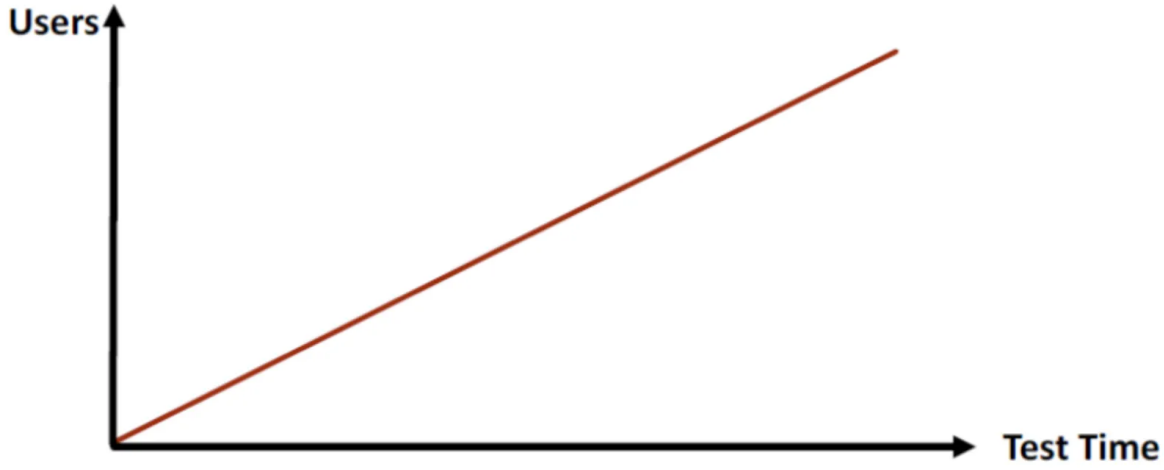

Figure 2.3. Stress Testing

Stress testing is performed to evaluate the system's behavior when the load exceeds the threshold, to find the breaking point and assess the system's ability to recover.

Spike Testing

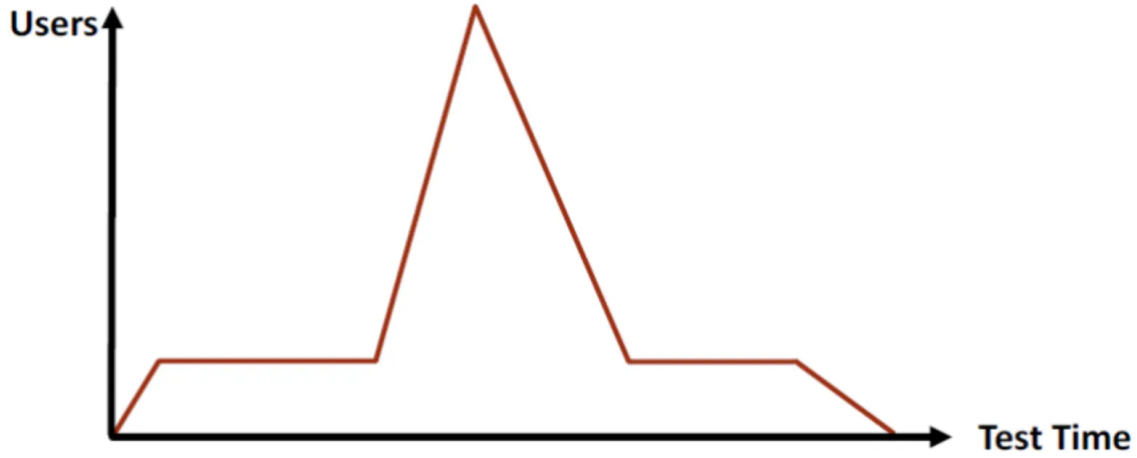

Figure 2.4. Spike Testing

Spike testing is performed to evaluate the system's behavior when the load **suddenly increases for a short period**. Some real-world scenarios that require spike testing:

- E-commerce platforms like Shopee, Lazada, etc., have many large flash sales on holidays and special days.
- Course registration systems for university students.
- Systems for selling football tickets, concert tickets, etc.
- Live streaming systems.

Endurance Testing (Soak Testing)

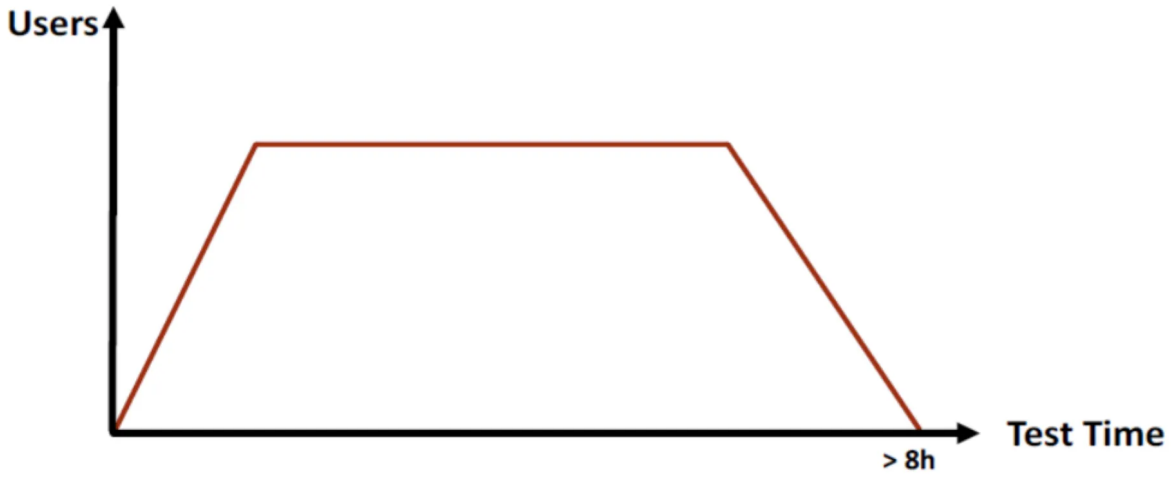

Figure 2.5. Soak Testing

Endurance testing is performed over a long period to evaluate the system's resource usage (memory). Typically, it runs at 70-80% of the system's threshold for more than 8 hours.

Scalability Testing

Scalability testing (or Capacity testing) is a type of testing that aims to evaluate the application's ability to handle load as the number of users or transactions increases. This type of testing helps collect the necessary metrics to forecast and determine the appropriate hardware configuration, ensuring the system can meet larger usage demands in the future.

The main purpose of this testing is to support **system sizing** and to check the application's scalability as its scale increases.

Scalability testing can be considered the **next step** after load testing when a business needs to prepare for growth and expand its user base in the future.

Volume Testing

Volume testing is a type of performance testing that aims to evaluate the software or application's ability to handle a **very large amount of data** in the database.

The main goal is to check if the system remains stable, fast, and error-free when accessing, processing, reporting, or synchronizing with **huge amounts of data**.

- **Example:** Testing the report generation feature when the database has millions of records, or checking the data synchronization speed between large systems.

2.2.4 Common System Performance Issues

Most performance issues revolve around speed, response time, load time, and poor scalability. Speed is often one of the most important attributes of an application. A slow application wastes time, reduces user satisfaction with the system, and can lead to the loss of potential users. Performance testing is done to ensure the application runs fast enough to attract and maintain user interest and satisfaction.

Below is a list of some common performance issues, which also highlights that speed is the most common factor:

- ***Long load time*:** Load time is typically the initial time it takes for an application to launch. This should generally be kept to a minimum. While some applications cannot load in under a minute, a load time of a few seconds is ideal.
- ***Slow response time*:** Response time is the time it takes from when a user inputs data into the application until the application provides a response to that input. Generally, this should be very fast. Again, if users have to wait too long, they will lose interest.
- ***Poor scalability*:** A software product has poor scalability if it cannot handle the expected number of users or if it does not meet the needs of a sufficient range of users. Load testing must be performed to ensure the application can handle the anticipated number of users.
- ***Bottlenecks*:** These are obstacles in the system that degrade the overall system performance. A bottleneck occurs when coding errors or hardware issues cause a drop in throughput under a certain load. Bottlenecks are often caused by a faulty piece of code. The key to fixing the problem is to perform bottleneck testing to find the section of code causing the slowdown and find a solution. Some common performance bottlenecks are: CPU, memory, network, operating system, and hard drive.

2.2.5 Common Metrics in Performance Testing

Depending on the scope of the test and the type of system being tested *(e.g., a web application or a mobile application)*, the measurement parameters or information to be collected and monitored will vary. Below are the general parameters/information often collected and monitored during performance testing:

- **Processor usage** – The amount of time the processor (CPU) spends executing processing threads.
- **Memory usage** – The amount of temporary memory (RAM) used to execute requests. This helps evaluate system performance for optimization.
- **Disk I/O** – The amount of time the disk is busy performing a read or write request.
- **Disk queue length** – The average number of read and write requests queued for the selected disk over a period of time.
- **Bandwidth** – Shows the number of bits per second (bits/s – representing network speed) used during the test.
- **Network output queue length** – The length of the output queue of packets. A queue length greater than one indicates delays and congestion.
- **Response time** – The time from when a user makes a request until the first response is received from the system (server).
- **Throughput** – Measures the number of requests per unit of time, usually seconds. Represents the **total number of requests/unit of time** over a period.
- **Hits per second** – The number of hits/requests sent to the server each second. Often measured during load testing.
- **Thread counts** – The number of currently running and active threads indicates the "health" of the system.

2.3 Research Performance Testing Tools & Compare

2.3.1 Overview of Performance Testing Tools

- **JMeter**: JMeter is an open-source performance testing tool developed by Apache. It is primarily used to test the durability, performance, and load capacity of web applications, APIs, and servers. JMeter is open-source software written in Java. It was originally created by Stefano Mazzocchi and later redesigned by Apache to improve its graphical user interface (GUI) and add functional testing capabilities. The tool supports many protocols like HTTP, FTP, JDBC, and SOAP, and can be extended with plugins. JMeter is popular in the community for its flexibility and its ability to simulate thousands of virtual users.

Figure 2.6. JMeter Logo

- **k6**: k6 is an open-source performance testing tool built with Go, using JavaScript for writing test scripts. It focuses on simplifying the testing process for developers, offering a command-line interface (CLI) and easy integration into CI/CD pipelines. The tool is centered on load testing for APIs, microservices, and web applications, with advantages like being lightweight, fast, and resource-efficient. k6 stands out with its strong developer community and detailed documentation, making it suitable for projects that require automation and scalability.

Figure 2.7. k6 Logo

- **LoadRunner**: LoadRunner is a commercial performance testing tool developed by Micro Focus. It is designed to simulate thousands of virtual users to evaluate the performance

and load capacity of applications and systems. The tool supports many protocols like HTTP, Web Services, SAP, Oracle, and Citrix, and provides detailed test script recording and playback. LoadRunner is known for its advanced analysis features and integration with project management tools, but it requires licensing fees and significant hardware resources. It is suitable for large enterprises that need large-scale testing and in-depth analysis.

Figure 2.8. LoadRunner Logo

- **NeoLoad**: NeoLoad is a commercial performance testing tool developed by Neotys. It focuses on simulating large loads to assess the performance of web applications, APIs, and mobile apps. With an intuitive GUI, NeoLoad makes it easy to record and design test scripts and offers cloud-based load distribution. The tool stands out for its strong CI/CD integration and detailed reporting but requires licensing fees. NeoLoad is suitable for medium to large businesses needing performance testing, especially in DevOps environments.

Figure 2.9. NeoLoad Logo

2.3.2 Detailed Comparison

Criteria	JMeter	k6	LoadRunner	NeoLoad
Tool Type	Open-source, GUI-based	Open-source, CLI-based	Commercial, GUI & script-based	Commercial, GUI-based

Criteria	JMeter	k6	LoadRunner	NeoLoad
Performance	Resource-intensive, but can be optimized with plugins and distributed load	High performance, optimized for large loads	High performance, but requires powerful hardware	Good performance, optimized for distributed load
Protocol Support	Diverse (HTTP, FTP, JDBC, SOAP, etc.), excellent support	Mainly HTTP, WebSocket, gRPC	Wide (HTTP, Web Services, SAP, Oracle, Citrix), enterprise-focused	Wide support (HTTP, SOAP, REST, etc.), mainly for web and APIs
Ease of Use	Easy with GUI, good for beginners	Easy for those who know JavaScript, difficult for beginners	Difficult for beginners, requires in-depth learning	Has a GUI, but users need to learn how to use it
DevOps Integration	Good integration but requires manual configuration	Easy integration with CI/CD	Strong enterprise integration, but complex	Good CI/CD integration, user-friendly
Reporting	Built-in, detailed via GUI and plugins	Customizable via CLI or k6 Cloud	In-depth reports but paid	Detailed reports, integrates with analysis tools
Community & Plugins	Large, many diverse plugins, strong support	Smaller, fewer plugins	Limited community, relies on Micro Focus support	Moderate community, relies on Neotys support
Cost	Completely free, unlimited	Free (local), paid for k6 Cloud	High cost, requires enterprise license	Paid

→ **Why we chose JMeter**: JMeter stands out for its excellent flexibility with multi-protocol support, a beginner-friendly GUI, and a large open-source community with many free plugins. JMeter offers a free and easily scalable solution through distributed loading, making it superior for projects with limited budgets or high flexibility requirements. Although it has a slight performance disadvantage compared to other tools, JMeter compensates with its detailed configuration options and diverse support, making it suitable for both beginners and experts.

2.4 A Detailed Look at Apache JMeter

2.4.1 JMeter's Overall Architecture

Apache JMeter is an open-source tool designed for performance and load testing of web applications and various other services. JMeter has a modular, plugin-based architecture that helps organize and simulate test flows.

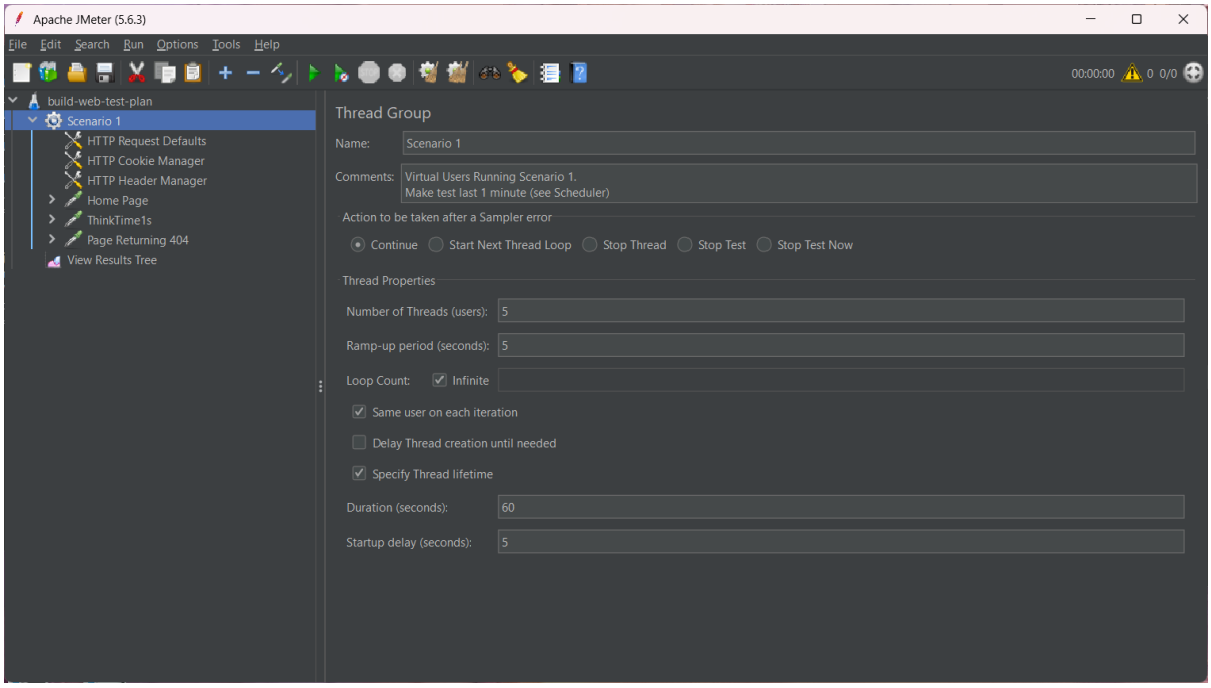

Figure 2.10. JMeter GUI

The main components in JMeter's architecture include:

- **Test Plan**: The root structure of every test, defining the entire test configuration and behavior.
- **Thread Group**: Represents virtual users, simulating a large number of users accessing the system.
- **Samplers**: Send specific requests to the system under test.
- **Logic Controllers**: Control the execution flow within a Thread Group.
- **Listeners**: Record and display test results.
- **Configuration Elements**: Provide default settings for Samplers.
- **Timers**: Add delays between requests.
- **Assertions**: Verify that the response matches expectations.
- **Pre/Post-Processors**: Process data before or after a request is sent.

These components can be nested in a hierarchical structure within a Test Plan to create complex test scenarios.

2.4.2 Thread Group

A Thread Group is the primary element for configuring how users are simulated. Each Thread Group can be configured with the following settings:

- **Number of Threads (users)**: The number of virtual users to simulate.

- **Ramp-Up Period (seconds)**: The time it takes to start all the virtual users.
- **Loop Count**: The number of times each user will repeat the test script.

Thread Groups also allow you to configure what happens on an error (e.g., stop the test, skip the sampler) and can contain many child elements like Samplers, Timers, and Assertions.

For example, if you configure 50 threads with a 10-second ramp-up and a loop count of 2, JMeter will start 5 threads every second, and each user will send their requests twice.

2.4.3 Sampler

A Sampler is responsible for sending different types of requests to the system being tested. Each Sampler corresponds to a specific protocol or action.

Some common types of Samplers include:

- **HTTP Request**: Sends HTTP/HTTPS requests to a web server.
- **JDBC Request**: Sends SQL queries to a database via JDBC.
- **FTP Request**: Sends requests to upload or download files from an FTP server.
- **SOAP/XML-RPC Request**: Sends requests to SOAP-based web services.
- **Java Request**: Allows you to directly call custom Java classes.
- **JMS Request**: Used to send/receive messages through a JMS system.

Samplers are crucial as they determine what system is being tested and how.

2.4.4 Listener

A Listener is a component that helps collect, record, and display test results. Listeners can write to a log, display real-time reports, or save results to a file for later analysis.

Some typical Listeners are:

- **View Results Tree**: Shows the details of every single request and response.
- **Summary Report**: Provides aggregate statistics like the number of requests, average response time, error rate, etc.
- **Aggregate Report**: Displays advanced aggregate information like standard deviation, response time percentiles, etc.
- **Graph Results**: Plots test data on a graph.
- **View Results in Table**: Displays a list of requests in a table format.

Choosing the right Listener makes analyzing system performance more intuitive and accurate.

2.4.5 Timers, Assertions, Pre-Processors, and Post-Processors

1. **Timers**

Timers are used to add delays between requests to simulate realistic user behavior. Some commonly used Timers are:

- **Constant Timer**: Adds a fixed delay.
- **Uniform Random Timer**: Adds a random delay with a uniform distribution.
- **Gaussian Random Timer**: Adds a random delay with a normal (Gaussian) distribution.
- **Synchronizing Timer**: Pauses threads to send requests all at once.

2. **Assertions**

Assertions are used to verify that the response from the server is correct and valid. Some common Assertions include:

- **Response Assertion**: Checks the response content or status code.
- **Duration Assertion**: Checks that the response time does not exceed a specified value.
- **Size Assertion**: Checks the size of the response.
- **JSON/XML Assertion**: Checks the structure and data of JSON/XML in the response.

Assertions help evaluate the correctness of the system, not just its performance.

3. **Pre-Processors**

A Pre-Processor is executed before a Sampler runs. It is often used to:

- Prepare input data.
- Generate dynamic tokens, headers, or values.
- Call functions from BeanShell or JSR223 for custom logic.

An example is "User Defined Variables," which sets up values to be used in subsequent requests.

4. **Post-Processors**

A Post-Processor is executed after a Sampler sends a request. It is used to:

- Extract data from the response to use in a later step.
- Save data, write to a log, or process the response string.
- The most popular are the **Regular Expression Extractor**, **JSON Extractor**, and **XPath Extractor**.